گزارش کارآموزی

طراحی و پیاده‌سازی یک سیستم پایگاه داده NoSQL

برای یک شرکت تاکسی اینترنتی (مبتنی بر MongoDB)

دانشجو: مهرانا کریمیان

استاد: بابازاده

شهریور ۱۴۰۵

فهرست مطالب

فهرست مطالب 2

فصل ۱: مقدمه 4

۱-۱ بیان مسئله 4

۱-۲ اهداف پروژه 4

۱-۳ محدوده‌ی پروژه 4

فصل ۲: مبانی نظری 5

۲-۱ پایگاه داده رابطه‌ای در برابر NoSQL 5

۲-۲ انواع پایگاه داده NoSQL 5

۲-۳ چرا NoSQL برای سیستم تاکسی اینترنتی؟ 5

۲-۴ چرا MongoDB؟ 6

۲-۵ قضیه CAP و جایگاه MongoDB 6

۲-۶ مدل سازگاری BASE در برابر ACID 6

۲-۷ مقایسه‌ی تطبیقی SQL و NoSQL برای این کاربرد 7

فصل ۳: تحلیل نیازمندی‌ها و الگوهای کوئری 7

۳-۱ الگوهای مربوط به مسافر 7

۳-۲ الگوهای مربوط به راننده 7

۳-۳ الگوهای مربوط به سفر، پرداخت و امتیاز 8

فصل ۴: طراحی پایگاه داده 8

۴-۱ مجموعه‌های (Collections) اصلی 8

۴-۲ تصمیمات Embedding در برابر Referencing 8

۴-۳ ساختار Schemaها 9

۴-۴ ایندکس‌های کلیدی 9

فصل ۵: پیاده‌سازی 10

۵-۱ معماری کلی سیستم 10

۵-۲ فناوری‌های استفاده‌شده 11

۵-۳ ساختار پروژه 11

۵-۴ نمونه پیاده‌سازی: کوئری نزدیک‌ترین راننده 11

۵-۵ فهرست کامل API 12

فصل ۶: تست و اجرا 12
۶-۱ محیط اجرا 12
۶-۲ سناریوهای تست انجام‌شده 12
۶-۳ نتایج 13
فصل ۷: نتیجه‌گیری و پیشنهادات 13
۷-۱ جمع‌بندی 13
۷-۲ چالش‌ها 13
۷-۳ پیشنهادات برای توسعه آینده 13
پیوست: سوالات متداول جلسه دفاع 14
۱. چرا برای این پروژه **SQL** را انتخاب نکردید؟ 14
۲. چرا **vehicle** را داخل سند **Driver** قرار دادید ولی **Trip** را جدا نگه داشتید؟ 14
۳. اگر تعداد رانندگان به میلیون‌ها برسد، سیستم چه تغییری نیاز دارد؟ 14
۴. ایندکس **2dsphere** دقیقاً چه کاری انجام می‌دهد؟ 14
۵. تضمین می‌کنید داده‌ها هیچ‌وقت ناسازگار نمی‌شوند؟ 15
۶. چرا میانگین امتیاز راننده را از قبل محاسبه و ذخیره کردید (**Denormalization**)؟ 15
۷. تفاوت **Mongoose** با درایور خام **MongoDB** چیست؟ 15
۸. اگر می‌خواستید بخش پیام‌رسانی بین راننده و مسافر را اضافه کنید، چه تغییری می‌دادید؟ 15
منابع 15

فصل ۱: مقدمه

۱-۱ بیان مسئله

شرکت‌های تاکسی اینترنتی امروزه بخش مهمی از زیرساخت حمل‌ونقل شهری را تشکیل می‌دهند. این سیستم‌ها باید در هر لحظه با حجم بالایی از داده‌های در حال تغییر سروکار داشته باشند: موقعیت مکانی صدها یا هزاران راننده، درخواست‌های سفر مسافران، و تاریخچه‌ی تراکنش‌های مالی. طراحی صحیح پایگاه داده در چنین سیستمی، تأثیر مستقیمی بر سرعت پاسخ‌گویی، مقیاس‌پذیری، و تجربه‌ی کاربری دارد.

هدف این کارآموزی، طراحی و پیاده‌سازی یک نمونه‌ی عملی از پایگاه داده‌ی غیررابطه‌ای (NoSQL) برای یک سیستم تاکسی اینترنتی است؛ با تمرکز بر مدل‌سازی داده متناسب با الگوهای واقعی دسترسی به داده (Query Patterns)، به‌جای رویکرد سنتی نرمال‌سازی رابطه‌ای.

۱-۲ اهداف پروژه

- آشنایی عملی با مفاهیم پایگاه‌های داده NoSQL و تفاوت آن با پایگاه‌های داده رابطه‌ای
- طراحی مدل داده بر اساس الگوهای کوئری واقعی سیستم
- پیاده‌سازی یک API کاربردی با Node.js، Express و MongoDB
- پیاده‌سازی کوئری‌های جغرافیایی (Geospatial) برای یافتن نزدیک‌ترین راننده
- مستندسازی فرآیند طراحی و تصمیمات معماری

۱-۳ محدوده‌ی پروژه

به‌منظور تمرکز بر مفاهیم اصلیِ طراحی پایگاه داده NoSQL، برخی از قابلیت‌های تجاری سیستم‌های واقعی تاکسی اینترنتی از محدوده‌ی این پروژه خارج نگه داشته شدند، از جمله: تخمین قیمت پیش از سفر (Fare Estimation)، سیستم کیف پول و کد تخفیف، و پیام‌رسانی بین راننده و مسافر. این محدودسازی امکان تمرکز عمیق‌تر بر طراحی داده و کوئری‌های اصلی را فراهم کرده است.

فصل ۲: مبانی نظری

۲-۱ پایگاه داده رابطه‌ای در برابر NoSQL

پایگاه‌های داده رابطه‌ای (SQL) داده را در جداول با ساختار ثابت (Schema) ذخیره می‌کنند و روابط بین جداول را از طریق کلید خارجی (Foreign Key) و عملیات پیوند (Join) مدیریت می‌کنند. این مدل برای داده‌های با ساختار ثابت و روابط پیچیده مناسب است، اما در مقیاس بسیار بزرگ و در سناریوهایی با نرخ نوشتن بالا، می‌تواند به گلوگاه عملکردی تبدیل شود.

در مقابل، پایگاه‌های داده NoSQL (Not Only SQL) طیف گسترده‌ای از مدل‌های داده را شامل می‌شوند که ساختار سخت‌گیرانه‌ی جدولی را کنار گذاشته و در عوض بر مقیاس‌پذیری افقی، انعطاف‌پذیری Schema، و عملکرد بالا در عملیات خواندن/نوشتن تمرکز دارند.

۲-۲ انواع پایگاه داده NoSQL

نوع	توضیح	نمونه
سندمحور (Document)	ذخیره داده به‌صورت اسناد JSON/BSON با ساختار انعطاف‌پذیر	MongoDB, CouchDB
کلید-مقدار (Key-Value)	ذخیره ساده‌ی جفت کلید و مقدار، بسیار سریع	Redis, DynamoDB
ستون‌گرا (Column-family)	بهینه برای نوشتن حجیم و توزیع‌شده	Cassandra, HBase
گراف (Graph)	مدل‌سازی روابط پیچیده بین موجودیت‌ها	Neo۴j

۲-۳ چرا NoSQL برای سیستم تاکسی اینترنتی؟

- حجم و سرعت بالای نوشتن داده: به‌روزرسانی مکرر موقعیت لحظه‌ای رانندگان
- نیاز به کوئری‌های جغرافیایی سریع که MongoDB به‌صورت بومی با ایندکس ۲dsphere پشتیبانی می‌کند

- مقیاس‌پذیری افقی ساده‌تر نسبت به پایگاه‌های داده رابطه‌ای در مواجهه با رشد سیستم
- انعطاف در ساختار داده، بدون نیاز به Migration های پیچیده هنگام تغییر نیازمندی‌ها

۲-۴ چرا MongoDB؟

از میان گزینه‌های NoSQL، MongoDB به‌عنوان دیتابیس اصلی این پروژه انتخاب شد زیرا مدل سندمحور آن با موجودیت‌های طبیعی این سیستم (کاربر، راننده، سفر) هم‌خوانی مستقیم دارد، از ایندکس‌گذاری جغرافیایی پیشرفته پشتیبانی می‌کند، و اکوسیستم بالغی برای توسعه با Node.js (از طریق کتابخانه‌ی Mongoose) در اختیار قرار می‌دهد.

۲-۵ قضیه CAP و جایگاه MongoDB

قضیه‌ی CAP بیان می‌کند که یک سیستم پایگاه داده‌ی توزیع‌شده نمی‌تواند هم‌زمان هر سه ویژگی سازگاری (Consistency)، دسترس‌پذیری (Availability) و تحمل خطای افراز شبکه (Partition Tolerance) را به‌طور کامل تضمین کند و باید در عمل بین این سه، بده‌بستان (Trade-off) داشته باشد. MongoDB به‌صورت پیش‌فرض در دسته‌ی سیستم‌های CP (Consistency + Partition Tolerance) قرار می‌گیرد؛ به این معنا که در پیکربندی Replica Set، نوشتن‌ها ابتدا روی گره اصلی (Primary) اعمال و سپس به گره‌های ثانویه (Secondary) تکرار می‌شوند. این ویژگی برای سیستم تاکسی اینترنتی اهمیت دارد، زیرا داده‌هایی مانند وضعیت یک سفر یا مبلغ پرداختی باید در تمام نقاط سیستم سازگار باشند.

۲-۶ مدل سازگاری BASE در برابر ACID

پایگاه‌های داده رابطه‌ای سنتی بر پایه‌ی اصول ACID (اتمی بودن، سازگاری، ایزوله بودن، پایداری) طراحی می‌شوند که تضمین قوی برای صحت تراکنش‌ها ارائه می‌دهد، اما در مقیاس بسیار بزرگ هزینه‌ی عملکردی دارد. بسیاری از پایگاه‌های داده NoSQL، از جمله نسخه‌های توزیع‌شده‌ی MongoDB، به مدل BASE (اساساً در دسترس، وضعیت نرم، سازگاری نهایی) گرایش دارند. با این حال، باید توجه داشت که MongoDB از نسخه‌ی ۴ به بعد از تراکنش‌های چندسندی (Multi-document Transactions) نیز با تضمین‌های ACID پشتیبانی می‌کند، که در این پروژه برای عملیات حساس مانند پایان سفر و ثبت هم‌زمان پرداخت، قابل استفاده است، هرچند برای ساده‌سازی، در نسخه‌ی فعلی از آن استفاده نشده است.

۲-۷ مقایسه‌ی تطبیقی SQL و NoSQL برای این کاربرد

معیار	پایگاه داده رابطه‌ای (SQL)	MongoDB (NoSQL)
مدل داده	جدول با ستون‌های ثابت	سند JSON/BSON با ساختار انعطاف‌پذیر
روابط	از طریق Join و کلید خارجی	از طریق Embedding یا Reference
کوئری مکانی	نیازمند افزونه (مثل PostGIS)	پشتیبانی بومی با ایندکس ۲dsphere
مقیاس‌پذیری	عمدتاً عمودی (Vertical)	افقی (Horizontal) با Sharding
انعطاف Schema	تغییر ساختار پرهزینه (Migration)	افزودن فیلد جدید بدون توقف سرویس

این مقایسه نشان می‌دهد در حالی‌که پایگاه داده رابطه‌ای برای داده‌های با ساختار ثابت و روابط پیچیده‌ی چندسویه انتخاب مناسبی است، برای سیستمی با نرخ نوشتن بالا، نیاز شدید به کوئری‌های مکانی، و رشد سریع حجم داده مانند تاکسی اینترنتی، مدل سندمحور MongoDB برتری عملی دارد.

فصل ۳: تحلیل نیازمندی‌ها و الگوهای کوئری

برخلاف رویکرد سنتی طراحی پایگاه داده رابطه‌ای که با نرمال‌سازی آغاز می‌شود، طراحی مؤثر در NoSQL باید بر پایه‌ی «الگوهای دسترسی به داده» (Query Patterns) صورت گیرد. در این فصل، مهم‌ترین عملیاتی که سیستم باید پشتیبانی کند فهرست شده‌اند.

۳-۱ الگوهای مربوط به مسافر

- ثبت‌نام و مشاهده‌ی پروفایل مسافر
- ثبت درخواست سفر جدید با مبدأ و مقصد
- مشاهده وضعیت لحظه‌ای سفر جاری
- مشاهده تاریخچه‌ی سفرهای گذشته
- ثبت امتیاز و نظر برای راننده پس از پایان سفر

۳-۲ الگوهای مربوط به راننده

- ثبت‌نام راننده به همراه اطلاعات وسیله نقلیه
- تغییر وضعیت آنلاین/آفلاین
- به‌روزرسانی مکرر موقعیت مکانی لحظه‌ای

- یافتن نزدیک‌ترین رانندگان آنلاین به یک مختصات مشخص (مهم‌ترین کوئری سیستم)
- پذیرفتن یا رد کردن درخواست سفر
- مشاهده تاریخچه سفرها و درآمد

۳-۳ الگوهای مربوط به سفر، پرداخت و امتیاز

- ایجاد سفر با وضعیت اولیه و تغییر وضعیت در طول چرخه‌ی حیات سفر
- محاسبه و ثبت هزینه نهایی سفر
- ثبت تراکنش پرداخت و مشاهده تاریخچه آن
- ثبت امتیاز و محاسبه میانگین امتیاز راننده

این ۲۰ الگوی کوئری، مبنای اصلی تصمیمات طراحی Schema در فصل بعد قرار گرفته‌اند.

فصل ۴: طراحی پایگاه داده

۴-۱ مجموعه‌های (Collections) اصلی

بر اساس تحلیل فصل قبل، پنج مجموعه‌ی اصلی طراحی شدند:

Collection	توضیح
users	اطلاعات مسافران
drivers	اطلاعات رانندگان، شامل وسیله نقلیه و موقعیت مکانی
trips	سفرها؛ موجودیت مرکزی سیستم
payments	تراکنش‌های پرداخت
ratings	امتیازها و نظرات

۴-۲ تصمیمات Embedding در برابر Referencing

یکی از مهم‌ترین تصمیمات در طراحی داده‌ی NoSQL، انتخاب بین جای‌گذاری داده‌های مرتبط درون یک سند (Embedding) یا نگه‌داشتن آن‌ها به‌صورت جدا و ارجاع با شناسه (Referencing) است. جدول زیر تصمیمات گرفته‌شده در این پروژه و دلیل هرکدام را نشان می‌دهد:

رابطه	نوع	دلیل
Driver ↔ Vehicle	Embed	رابطه یک‌به‌یک؛ همواره با هم خوانده می‌شوند
User ↔ Trips	Reference	رابطه یک‌به‌چند نامحدود؛ جلوگیری از رشد بی‌رویه سند
Driver ↔ Trips	Reference	نیاز به کوئری مستقل از دو سمت (مسافر و راننده)
Trip ↔ Payment	Reference	رخداد مالی مستقل؛ امکان گزارش‌گیری جداگانه
Trip ↔ Rating	Reference	نیاز به کوئری مستقل روی نظرات هر راننده
Driver.rating.avg	Denormalized	امکان خواندن سریع میانگین امتیاز بدون Aggregate

۴-۳ ساختار Schemaها

نمونه‌ای از ساختار Schema مربوط به راننده (Driver)، شامل وسیله نقلیه جای‌گذاری‌شده و فیلد موقعیت مکانی به فرمت GeoJSON:

```
{
  name: String,
  phone: String,
  vehicle: {              // Embedded
    plateNumber: String,
    model: String,
    color: String
  },
  status: "online" | "offline" | "on_trip",
  currentLocation: {
    type: "Point",
    coordinates: [lng, lat]
  },
  rating: { avg: Number, count: Number }
}
```

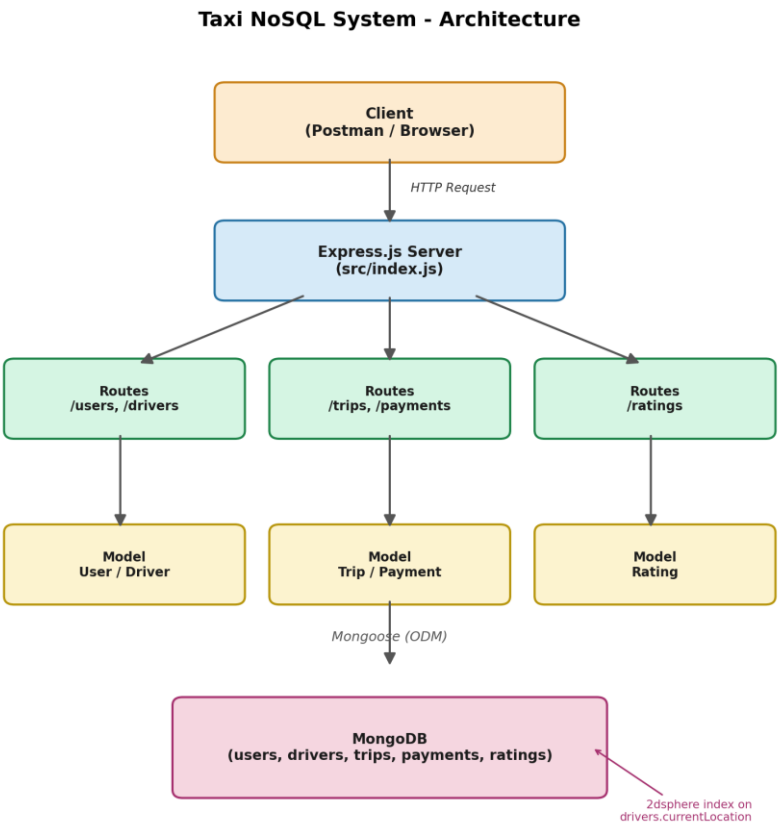
۴-۴ ایندکس‌های کلیدی

- drivers.currentLocation ⟶ ایندکس ۲dsphere (پایه‌ی کوئری نزدیک‌ترین راننده)
- trips.riderId و trips.driverId ⟶ تسریع کوئری تاریخچه سفرها
- users.phone و drivers.phone ⟶ ایندکس یکتا (Unique)

فصل ۵: پیاده‌سازی

۵-۱ معماری کلی سیستم

شکل زیر جریان یک درخواست را از سمت کلاینت تا رسیدن به پایگاه داده و بازگشت پاسخ نشان می‌دهد. هر لایه مسئولیت مشخصی دارد: لایه‌ی Routes مسیر HTTP را به عملیات مربوطه نگاشت می‌کند، لایه‌ی Models (از طریق Mongoose) ساختار داده و اعتبارسنجی را مدیریت می‌کند، و در نهایت MongoDB داده را به‌صورت پایدار ذخیره می‌کند.



۵-۲ فناوری‌های استفاده‌شده

فناوری	نقش در پروژه
Node.js	محیط اجرای سمت سرور
Express.js	فریم‌ورک ساخت REST API
MongoDB	پایگاه داده NoSQL اصلی
Mongoose	لایه‌ی نگاشت شیء به سند (ODM) برای تعریف Schema و کوئری‌نویسی
dotenv	مدیریت متغیرهای محیطی
Faker.js	تولید داده‌ی تستی برای Seed

۵-۳ ساختار پروژه

```
taxi-nosql/
├── src/
│   ├── config/db.js        # اتصال به MongoDB
│   ├── models/              # تعریف Schemaها (User, Driver, Trip, Payment, Rating)
│   ├── routes/              # مسیرهای API
│   ├── scripts/seed.js      # تولید داده‌ی تستی
│   └── index.js             # فایل اصلی اجرای سرور
├── .env.example
├── package.json
└── README.md
```

۵-۴ نمونه پیاده‌سازی: کوئری نزدیک‌ترین راننده

مهم‌ترین و چالش‌برانگیزترین کوئری این سیستم، یافتن نزدیک‌ترین رانندگان آنلاین به یک مختصات مشخص است. این کوئری با استفاده از عملگر $near در MongoDB و بر پایه‌ی ایندکس ۲dsphere پیاده‌سازی شده است:

```
Driver.find({
  status: "online",
  currentLocation: {
    $near: {
      $geometry: { type: "Point", coordinates: [lng, lat] },
      $maxDistance: 5000   // بر حسب متر
    }
  }
}).limit(10);
```

۵-۵ فهرست کامل API

متد	مسیر	توضیح
POST	users/	ثبت‌نام مسافر
GET	users/:id/trips/	تاریخچه سفرهای مسافر
POST	drivers/	ثبت‌نام راننده
PATCH	drivers/:id/location/	به‌روزرسانی موقعیت راننده
GET	drivers/nearby/	یافتن نزدیک‌ترین رانندگان آنلاین
POST	trips/	ثبت درخواست سفر
PATCH	trips/:id/accept/	پذیرفتن سفر توسط راننده
PATCH	trips/:id/complete/	پایان سفر و ثبت هزینه
POST	payments/	ثبت پرداخت
POST	ratings/	ثبت امتیاز راننده

فصل ۶: تست و اجرا

۶-۱ محیط اجرا

پروژه در محیط لوکال با نصب Node.js و MongoDB Community Server اجرا و تست شد. پس از اجرای دستور npm install برای نصب وابستگی‌ها، با اسکریپت Seed حدود ۲۰ مسافر، ۱۵ راننده، و ۳۰ سفر تکمیل‌شده به‌همراه پرداخت و امتیاز مرتبط، به‌صورت خودکار در پایگاه داده ایجاد شد.

۶-۲ سناریوهای تست انجام‌شده

- تست اتصال سرور Express به پایگاه داده MongoDB
- تست کوئری نزدیک‌ترین راننده با مختصات نمونه
- تست چرخه‌ی کامل یک سفر: ثبت درخواست ← پذیرش ← شروع ← پایان ← پرداخت ← امتیازدهی
- بررسی صحت به‌روزرسانی خودکار میانگین امتیاز راننده پس از ثبت امتیاز جدید

۶-۳ نتایج

تمامی سناریوهای تست با موفقیت اجرا شدند. کوئری نزدیک‌ترین راننده با بهره‌گیری از ایندکس ۲dsphere، پاسخ سریع و صحیحی را حتی با افزایش تعداد رانندگان ثبت‌شده در پایگاه داده ارائه داد که تأییدکننده‌ی صحت انتخاب طراحی مکانی است.

فصل ۷: نتیجه‌گیری و پیشنهادات

۷-۱ جمع‌بندی

در این کارآموزی، یک سیستم پایگاه داده NoSQL برای یک شرکت فرضی تاکسی اینترنتی طراحی و پیاده‌سازی شد. با تمرکز بر الگوهای واقعی دسترسی به داده به‌جای نرمال‌سازی سنتی، مدلی طراحی شد که هم از نظر عملکرد (به‌ویژه در کوئری‌های جغرافیایی) و هم از نظر مقیاس‌پذیری، برای این نوع کاربرد مناسب است.

۷-۲ چالش‌ها

- درک تفاوت مدل فکری طراحی NoSQL نسبت به رویکرد آموخته‌شده در پایگاه‌های داده رابطه‌ای
- تصمیم‌گیری در مورد میزان Denormalization بدون از دست دادن سازگاری داده
- پیکربندی صحیح ایندکس جغرافیایی برای عملکرد بهینه

۷-۳ پیشنهادات برای توسعه آینده

- افزودن سیستم تخمین قیمت پیش از سفر بر اساس فاصله و ترافیک
- پیاده‌سازی Sharding برای مقیاس‌پذیری در سطح واقعی صنعتی
- افزودن لایه‌ی کش (مانند Redis) برای موقعیت لحظه‌ای رانندگان با نرخ به‌روزرسانی بسیار بالا
- افزودن سیستم پیام‌رسانی real-time بین راننده و مسافر با WebSocket

پیوست: سوالات متداول جلسه دفاع

در این بخش، پرسش‌های محتمل استاد راهنما یا داوران به‌همراه پاسخ کوتاه و کلیدی هرکدام آورده شده است تا آمادگی جلسه‌ی دفاع تسهیل شود.

۱. چرا برای این پروژه SQL را انتخاب نکردید؟

چون سیستم تاکسی اینترنتی نیازمند نرخ نوشتن بسیار بالا (به‌روزرسانی مکرر موقعیت رانندگان)، کوئری‌های جغرافیایی سریع، و مقیاس‌پذیری افقی است؛ MongoDB این نیازها را به‌صورت بومی و با هزینه‌ی مهندسی کمتر پوشش می‌دهد.

۲. چرا vehicle را داخل سند Driver قرار دادید ولی Trip را جدا نگه داشتید؟

معیار اصلی تصمیم، الگوی دسترسی به داده بود، نه شباهت مفهومی. رابطه‌ی راننده و وسیله‌ی نقلیه یک‌به‌یک است و همیشه با هم خوانده می‌شود، بنابراین Embedding هزینه‌ی یک کوئری اضافه را حذف می‌کند. اما یک راننده می‌تواند صدها سفر داشته باشد؛ اگر سفرها را داخل سند راننده Embed می‌کردیم، سند به‌مرور بسیار بزرگ و کند می‌شد و کوئری مستقل روی سفرها (برای گزارش‌گیری یا مسافر) دشوار می‌گشت.

۳. اگر تعداد رانندگان به میلیون‌ها برسد، سیستم چه تغییری نیاز دارد؟

در آن مقیاس باید از Sharding (افراز افقی داده بر پایه‌ی یک کلید Shard مناسب، مثلاً منطقه‌ی جغرافیایی) استفاده کرد و موقعیت لحظه‌ای رانندگان را به یک فروشگاه حافظه‌محور مانند Redis منتقل کرد تا فشار نوشتن مکرر از MongoDB اصلی جدا شود.

۴. ایندکس ۲dsphere دقیقاً چه کاری انجام می‌دهد؟

این ایندکس داده‌های جغرافیایی به‌فرمت GeoJSON را در ساختاری بهینه برای جستجوی مکانی (مانند یافتن نزدیک‌ترین نقاط یا نقاط درون یک محدوده) سازمان‌دهی می‌کند. بدون این ایندکس، MongoDB برای هر کوئری مکانی باید تمام اسناد را به‌صورت خطی بررسی کند (Collection Scan) که در مقیاس بزرگ بسیار کند است.

۵. تضمین می‌کنید داده‌ها هیچ‌وقت ناسازگار نمی‌شوند؟

در طراحی فعلی، برخی عملیات (مانند پذیرفتن سفر و تغییر هم‌زمان وضعیت راننده) در دو نوشتن جدا انجام می‌شوند، نه در یک تراکنش اتمی؛ این یک محدودیت شناخته‌شده است. راه‌حل، استفاده از قابلیت Multi-document Transactions در MongoDB است که در توسعه‌ی آینده قابل افزودن است.

۶. چرا میانگین امتیاز راننده را از قبل محاسبه و ذخیره کردید (Denormalization)؟

چون این مقدار در هر نمایش پروفایل راننده خوانده می‌شود (خواندن پرتکرار) اما فقط پس از ثبت یک امتیاز جدید تغییر می‌کند (نوشتن کم‌تکرار). محاسبه‌ی زنده با Aggregate در هر خواندن، هزینه‌ی محاسباتی غیرضروری تحمیل می‌کرد.

۷. تفاوت Mongoose با درایور خام MongoDB چیست؟

درایور خام (mongodb) دسترسی مستقیم و کم‌سربار به دیتابیس می‌دهد اما هیچ اعتبارسنجی یا ساختار از پیش تعریف‌شده‌ای ارائه نمی‌کند. Mongoose یک لایه‌ی انتزاعی (ODM) روی آن اضافه می‌کند که امکان تعریف Schema، اعتبارسنجی خودکار، و روابط بین مدل‌ها (populate) را فراهم می‌سازد؛ که سرعت توسعه و اطمینان از صحت داده را افزایش می‌دهد.

۸. اگر می‌خواستید بخش پیام‌رسانی بین راننده و مسافر را اضافه کنید، چه تغییری می‌دادید؟

نیاز به یک لایه‌ی ارتباط بلادرنگ (مانند WebSocket یا Socket.IO) در کنار پایگاه داده بود؛ و از نظر مدل داده، یک Collection جدید به نام messages با Reference به tripId، برای نگهداری تاریخچه‌ی گفتگو طراحی می‌شد.

منابع

مستندات رسمی MongoDB — https://www.mongodb.com/docs/

مستندات رسمی Mongoose — https://mongoosejs.com/docs/

مستندات رسمی Express.js — https://expressjs.com/